

Benchmark for Android Devices

Student: Octavian Mirisan

Structura Sistemelor de Calcul

Universitatea Tehnica Cluj-Napoca

Continut

1.Introducere

- 1.1 Context
- 1.2 Obiective

2.Studiu Bibliografic

- 2.1 Aplicatiile de Benchmark si importanta lor
- 2.2 Evaluarea performantei CPU
- 2.3 Evaluarea performantei grafice (GPU)
- 2.4 Evaluarea memoriei RAM

3.Analiza

- 3.1 Masurarea performantei CPU
- 3.2 Masurarea performantei GPU
- 3.3 Masurarea performantei memoriei
- 3.4 Selectarea informatiilor despre hardware
- 3.5 Diagrama de UseCase
- 3.6 Proiectarea Interfetei Utilizator

4.Design

- 4.1 Diagrame (de pachete, de activitate, de clase)

5.Implementare

- 5.1 Structura pachetelor proiectului
- 5.2 Structura clasei MainActivity
- 5.3 Structura clasei CpuBenchmark
- 5.4 Structura clasei MemoryBenchmark
- 5.5 Structura clasei HardwareInfo

6.Testare si Validare

7.Concluzii

Bibliografie

Introducere

1.1 Context

În contextul dezvoltării rapide a dispozitivelor mobile, sistemul de operare Android a devenit unul dintre cele mai utilizate, oferind un ecosistem extins de aplicații și funcționalități. Performanța unui dispozitiv Android este esențială pentru experiența utilizatorului, afectând direct viteza de răspuns a aplicațiilor, cât și capacitatea de a rula sarcini intensive, precum jocurile sau aplicațiile de realitate augmentată [1].

Evaluarea și măsurarea performanței dispozitivelor Android sunt cruciale atât pentru dezvoltatorii de software cât și pentru producătorii de hardware, permitând optimizarea experienței de utilizare. Aplicațiile de benchmark sunt utilizate pentru a analiza diverse componente ale dispozitivelor, cum ar fi procesorul (CPU), unitatea de procesare grafică (GPU), memoria RAM și stocarea internă, oferind astfel informații valoroase despre performanța dispozitivelor [1][2]. Aceste aplicații permit utilizatorilor și dezvoltatorilor să identifice atât punctele forte, cât și posibile limitări ale dispozitivelor testate [2].

1.2 Obiective

Scopul acestui proiect este de a dezvolta o aplicație de benchmark pentru dispozitivele Android, destinată evaluării performanței componentelor hardware critice: CPU, GPU, memoria RAM. Aplicația va include teste specifice fiecărui component și va afișa rezultatele într-un format accesibil. Astfel, proiectul va oferi o evaluare detaliată a performanței dispozitivelor Android.

Obiectivele specifice ale proiectului includ:

1. Dezvoltarea testelor pentru procesor (CPU) pentru a măsura viteza de execuție a unor sarcini [1]
2. Evaluarea performanței grafice (GPU) prin teste grafice pentru a evalua capacitatea de redare a unui conținut grafic. [2][3]
3. Testarea memoriei RAM, pentru a determina viteza de accesare a datelor [4]
4. Interfața grafică și raportarea rezultatelor, care vor permite utilizatorului final să înțeleagă performanțele dispozitivelor [2]

2. Studiu Bibliografic

2.1 Aplicatii de benchmark si importanta lor

Aplicatiile de benchmark au un rol important in evaluarea performantei dispozitivelor mobile, oferind utilizatorilor si dezvoltatorilor o modalitate de a cuantifica performantele hardware. Benchmark-urile sunt esentiale pentru a compara dispozitivele si pentru a identifica modelele cele mai eficiente in rularea aplicatiilor complexe, de la gaming la vizualizarea multimedia sau aplicatii de productivitate [1][3].

Exista diverse aplicatii de benchmark pentru dispozitive Android, cele mai cunoscute fiind AnTuTu, Geekbench si 3DMark. Fiecare dintre acestea e conceputa pentru a evalua aspecte specifice ale performantei:

- AnTuTu ofera o evaluare globala a dispozitivului, masurand performantele CPU, GPU, memorie si interfata [1].
- Geekbench se axeaza pe performanta CPU, oferind rezultate pentru executia single-core si multi-core, esentiale pentru aplicatiile modern [2].
- 3DMark este specializat in evaluarea performantei grafice, furnizand scoruri bazate pe capacitatea GPU-ului de a reda scene complexe [3].

2.2 Evaluarea performantei CPU-ului

CPU-ul este componenta centrala a oricarui dispozitiv mobil, responsabila de executia instructiunilor si gestionarea fluxului de lucru a aplicatiilor. Aplicatiile de benchmark evalueaza performanta CPU-ului prin rularea unor seturi variate de instructiuni, cum ar fi simularea unor algoritmi matematici. Aceste teste ofera o masura obiectiva a vitezei si a eficientei CPU-ului, date utile pentru dezvoltatorii care doresc sa optimizeze aplicatiile pentru anumite modele de dispozitive [2][4].

2.3 Evaluarea performantei grafice (GPU)

GPU-ul e esential in aplicatiile ce necesita redarea grafica de inalta calitate. Aplicatiile de benchmark pentru GPU evalueaza capacitatea acestuia de a procesa texturi complexe si de a reda imagini fluide. Aceste teste sunt vitale pentru dispozitive orientate catre jocuri, realitate augmentata, unde grafica are un impact direct asupra experientei utilizatorului. [3][5]

2.4 Evaluarea memoriei RAM

Memoria RAM este utilizata pentru stocarea temporara a datelor si a instructiunilor necesare aplicatiilor in timpul executiei. Testele de performanta pentru memorie includ evaluarea vitezei de citire/scriere si a capacitatii de gestionare a datelor, fiind esentiale pentru a determina cat de rapid si eficient poate accesa si manipula un dispozitiv datele stocate [4][5].

3. Analiza

3.1 Masurarea Performantei CPU-ului

Pentru evaluarea performantei procesorului (CPU) in aplicatii de benchmark, este esential sa fie masurata eficienta in calcularea unor operatiuni matematice clasice. Testele selectate sunt reprezentative pentru sarcinile intensive de calcul si sunt folosite frecvent in evaluarea capacitatii de procesare:

1. Calcularea factorialului unui numar mare

Factorialul unui numar n (notat cu $n!$) reprezinta produsul tuturor numerelor intregi de la 1 pana la n inclusiv. Algoritmul de calcul poate fi realizat atat recursiv cat si iterativ, iar in aplicatiile de benchmark, timpul necesar pentru a calcula un factorial de ordin mare ofera informatii despre viteza procesorului in manipularea datelor si executia operatiunilor de multiplicare repetata. Factorialul are o crestere liniara in complexitate $[O(n)]$, deci este adecvat pentru teste de performanta in care se doreste observarea performantei CPU-ului.

2. Generarea termenilor unui sir Fibonacci

Sirul Fibonacci reprezinta o secventa numerica in care fiecare termen este suma celor doi termeni precedati de acesta. Formula generala pentru al n -lea termen dintr-un astfel de sir este $F(n) = F(n-1) + F(n-2)$. Algoritmul clasic de determinare al unui sir Fibonacci poate fi implementat recursiv, dar pentru testele de benchmark, implementarile iterative sau prin metoda matricala sunt mai eficiente si mai potrivite pentru a masura capacitatea CPU-ului de a lucra cu date in structuri repetitive.

3. Sortarea unui sir de elemente folosind diversi algoritmi de sortare

- **Bubble Sort:** reprezinta un algoritm de sortare simplu, avand o complexitate generala egala cu $O(n^2)$. Mecanismul de functionare al acestui algoritm este de a parcurge sirul de mai multe ori si de a interschimba elementele adiacente daca acestea se afla in ordinea gresita. Am ales acest algoritm deoarece este adecvat pentru testele de baza care solicita procesorul, intrucat necesita operatiuni repetate de comparare si interschimbare.
- **Quick Sort:** este un algoritm mai eficient din punct de vedere al timpului de executie decat Bubble Sort, avand o complexitate medie de $O(n \log n)$. Quick sort functioneaza prin a diviza sirul principal in subsiruri pe baza unui element ales aleator numit pivot, iar fiecare subsir este sortat recursiv. Timpul de executie al acestui algoritm ofera date despre capacitatea CPU-ului de a efectua operatiuni de partitionare si de recursivitate in mod eficient.

3.2 Masurarea performantei GPU

Evaluarea performantei GPU-ului este cruciala pentru aplicatii care implica redarea graficii complexe. Pentru un test de performanta al GPU-ului, am ales sa masuram timpul necesar pentru a reda un numar mare de obiecte 3D

- **Randarea unui numar mare de obiecte 3D**

GPU-ul este specializat in paralelizarea sarcinilor si este ideal pentru randarea obiectelor 3D complexe. Acest test implica crearea si randarea unui set de obiecte tridimensionale diverse, in mod implicit mii sau zeci de mii la numar, fiecare avand o geometrie diferita (ex. cuburi, sfere, piramide). Timpul necesar al GPU-ului pentru a termina randarea indica eficienta sa in gestionarea datelor grafice si in rulara shader-elor care definesc culorile si texturile obiectelor. Utilizarea shader-elor si calculelor de paralelism in renderizarea acestor obiecte reflecta capacitatea GPU-ului de a gestiona o incarcare grafica masiva.

3.3 Masurarea performantei memoriei

Evaluarea memoriei RAM se concentreaza pe viteza de acces si stabilitatea in gestionarea resurselor de memorie, doua aspecte critice pentru performanta generala a dispozitivului.

- 1. Testarea alocarii in memoria RAM a unor matrice de dimensiuni mari**

Alocarea si manipularea unor structuri mari de date, cum ar fi matricele, testeaza viteza si capacitatea RAM-ului. In acest test, se alocă succesiv matrice de dimensiuni mari si se masoara timpul necesar pentru fiecare operatiune. Aceasta abordare permite verificarea capacitatii memoriei RAM de a gestiona volume mari de date si ofera informatii despre viteza accesului la memorie.

3.4 Selectarea informatiilor despre hardware

Pentru a completa evaluarea performantei dispozitivului, este necesar sa se obtina si sa se afiseze informatii detaliate despre hardware-ul dispozitivului, precum modelul, versiunea sistemului de operare, capacitatea memoriei si spatiu de stocare, informatiile despre baterie si dimensiunea ecranului. Aceste informatii permit o intelegere completa a resurselor disponibile si a limitarilor hardware-ului si pot fi corelate cu performantele masurate ale CPU, GPU si memoriei pentru a oferi un context suplimentar.

1. Memorie RAM

- Capacitate totala RAM (in GB si MB)
- Cantitatea de memorie RAM disponibila in momentul evaluarii (GB si MB)
- Aceste date sunt importante pentru a intelege modul in care CPU-ul si aplicatiile utilizeaza memoria in timp real si pentru a evalua performanta dispozitivului in sarcinile intensive de calcul si randare

2. Stocare interna

- Capacitate totala a spatiului de stocare (GB si MB)
- Spatiul disponibil in momentul evaluarii (GB si MB)
- Informatiile despre stocare ofera detalii despre capacitatea dispozitivului de a gestiona volume mari de date si indica limitele pentru aplicatiile care necesita mult spatiu, cum ar fi aplicatiile multimedia si jocurile video

3. Informatii despre baterie

- Capacitatea bateriei (%) si starea generala de sanatate (e.g. Good, Overheat, Over Voltage)
- Starea bateriei este esentiala pentru a evalua autonomia si eficienta energetica a dispozitivului, mai ales in sarcinile intensive care consuma rapid bateria

4. Ecran

- Rezolutia ecranului (pixeli)
- Densitatea ecranului (dpi) si dimensiunea aproximativa a ecranului in inci
- Ecranul este o componenta hardware relevanta pentru performanta grafica (GPU) si permite aplicarea unei sarcini realiste, in special in testele de randare 3D si randare multimedia.

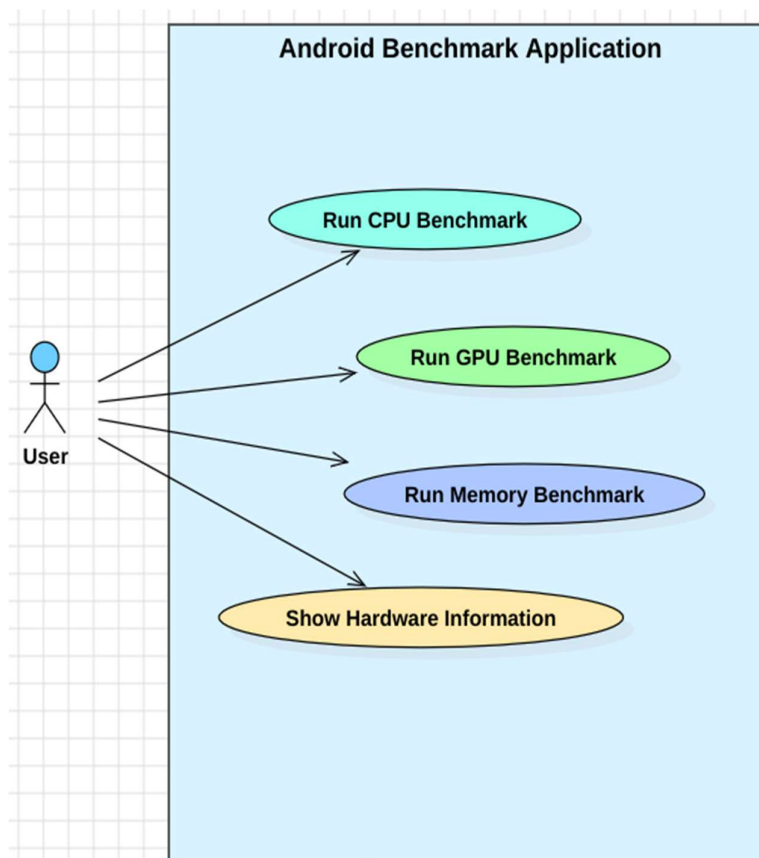
5. Informatii despre dispozitiv

- Modelul, brand-ul si arhitectura CPU-ului (ABI)
- Versiunea sistemului de operare (Android)

Aceste informatii definesc specificatiile de baza si sunt esentiale pentru a compara performantele intre diferite dispozitive

3.5 Diagrama de UseCase

Diagramele de Use Case sunt folosite pentru a ilustra interactiunea dintre utilizator si functionalitatile principale a aplicatiei.



1.User (Actor): actorul este utilizatorul aplicatiei, reprezentat prin simbolul unei figuri umane. Acesta initiaza toate actiunile posibile disponibile in aplicatie

2.Run CPU Benchmark: acesta este un use case prin care utilizatorul poate initia un test de performanta pentru procesorul dispozitivului mobil.

3.Run GPU Benchmark: prin acest use case, utilizatorul poate rula un test pentru GPU-ul dispozitivului. Acest test masoara performantele grafice, analizand puterea de procesare grafica si capacitatile GPU-ului.

4.Run Memory Benchmark: acest test permite utilizatorului sa evalueze performanta memoriei dispozitivului. Testul returneaza informatii despre viteza de citire/scriere si capacitatea memoriei.

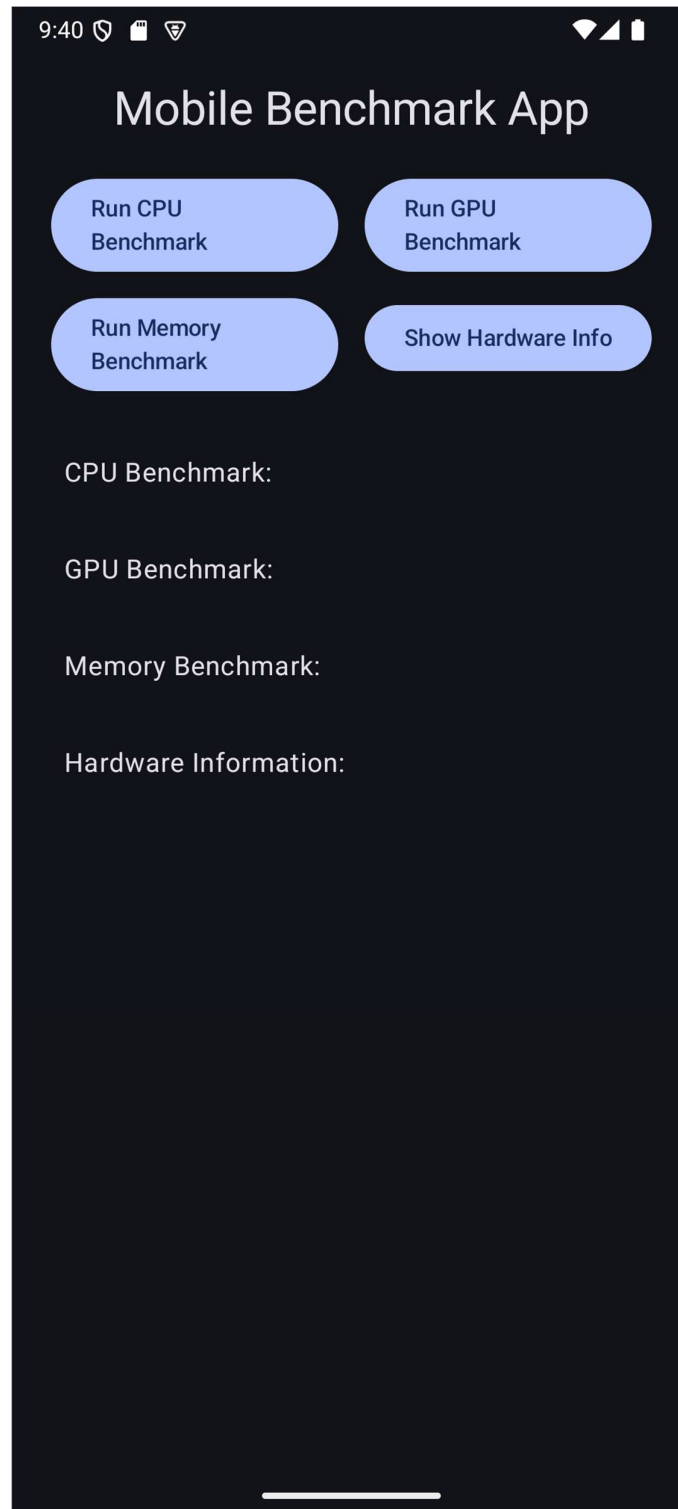
5. Show Hardware Information: prin acest use case, utilizatorul poate vizualiza informatii detaliate despre configuratia hardware a dispozitivului. Acesta include specificatiile si caracteristicile hardware, cum ar fi date legate de baterie, modelul telefonului, memoria RAM.

3.6 Proiectarea interfetei utilizator

Designul interfetei urmareste simplitatea si functionalitatea, avand o structura clara pentru accesul rapid la functiile aplicatiei:

Structura interfetei:

- Include butoane de testare a performantei procesorului (CPU), a placii grafice (GPU) precum si a memoriei dispozitivului mobil. Include si un buton cu ajutorul caruia se afiseaza specificatiile hardware ale dispozitivului.
- O sectiune dedicata pentru fiecare test, pentru afisarea rezultatelor testelor in timp real



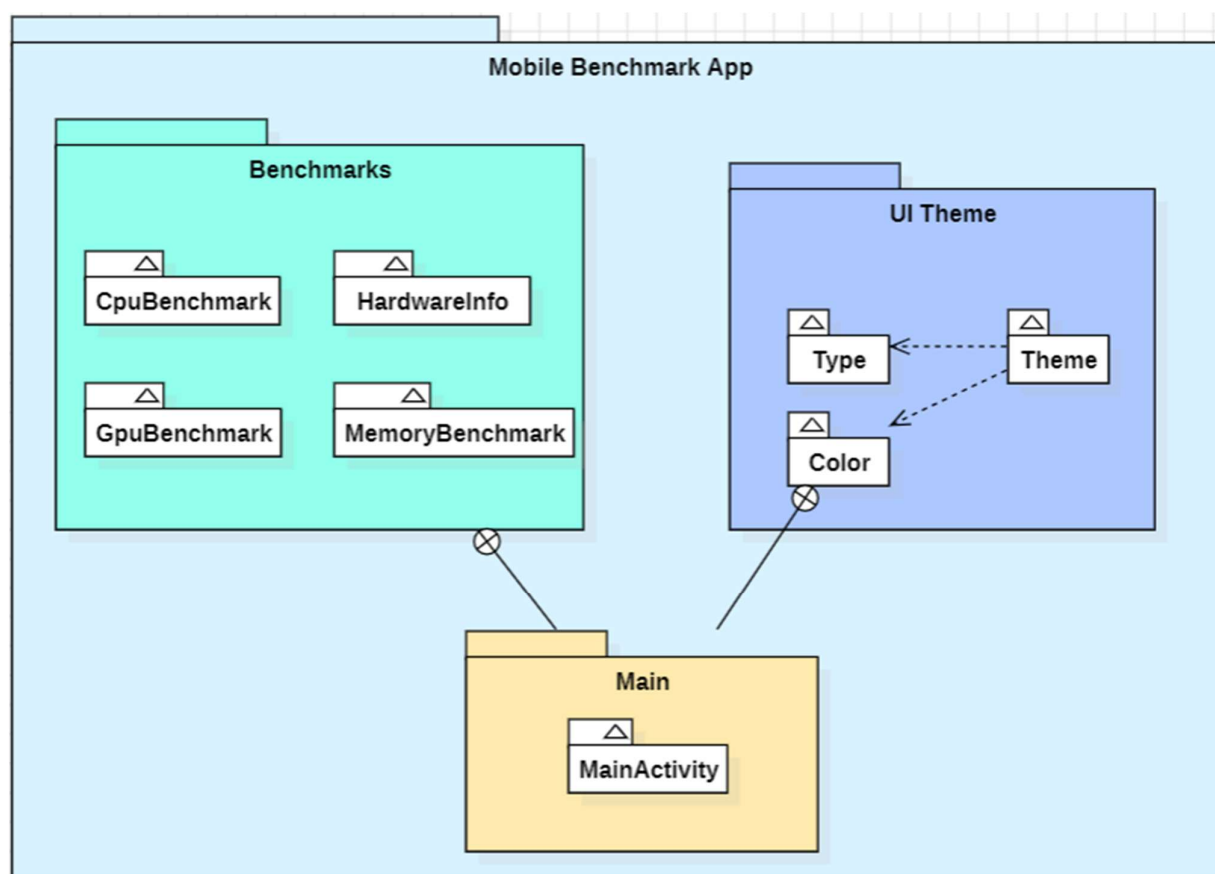
4. Design

4.1 Diagrame

4.1.1 Diagrama de Pachete

Aceasta diagrama aduce in prim plan o imagine de ansamblu a structurii aplicatiei evidentiind modul in care principalele pachete (module) sunt organizate si interactioneaza.

Scopul principal al acestui tip de diagrama este de a ilustra compartimentarea proiectului in module independente dar care comunica intre ele. Fiecare pachet reprezinta o clasa sau un set de clase cu functionalitati inrudite.



1. **MobileBenchmarkApp**: Este container-ul principal care grupeaza toate pachetele si clasele necesare pentru functionalitatea completa a aplicatiei
2. **UI Theme**: are ca scop principal gestionarea aspectului vizual al aplicatiei. Componentele care fac parte din acest pachet sunt:

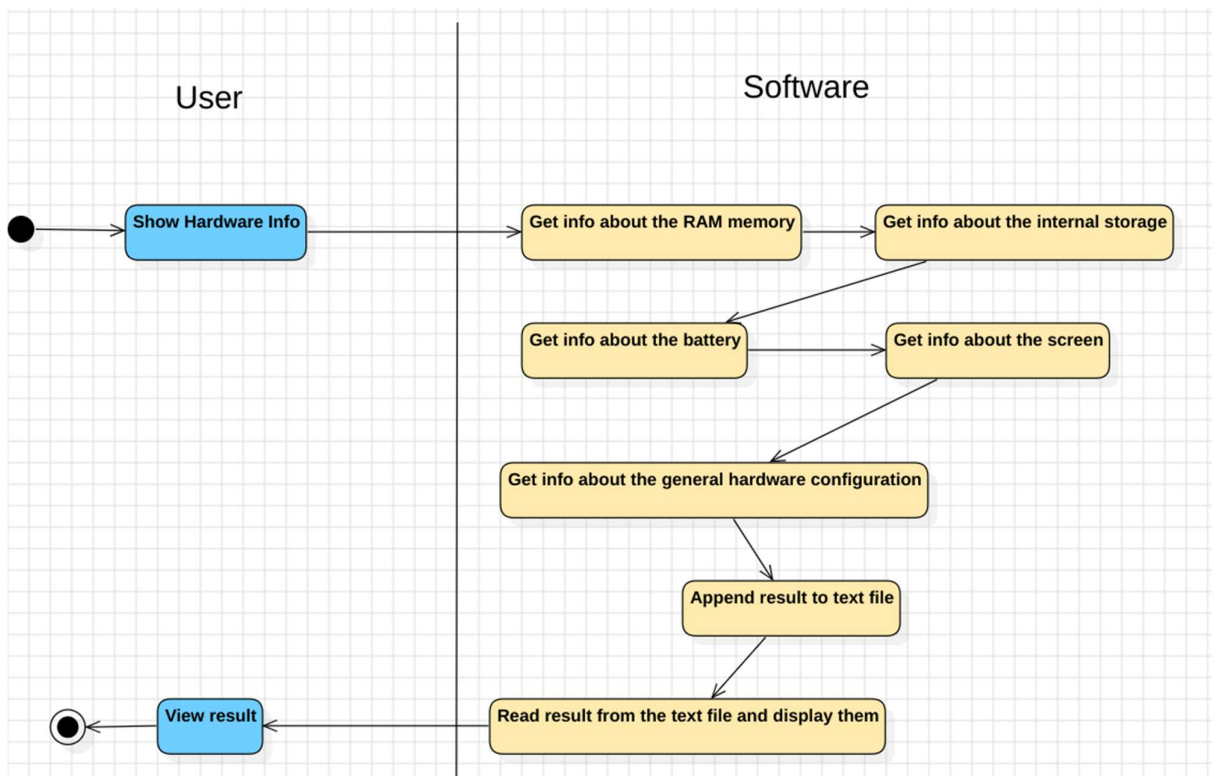
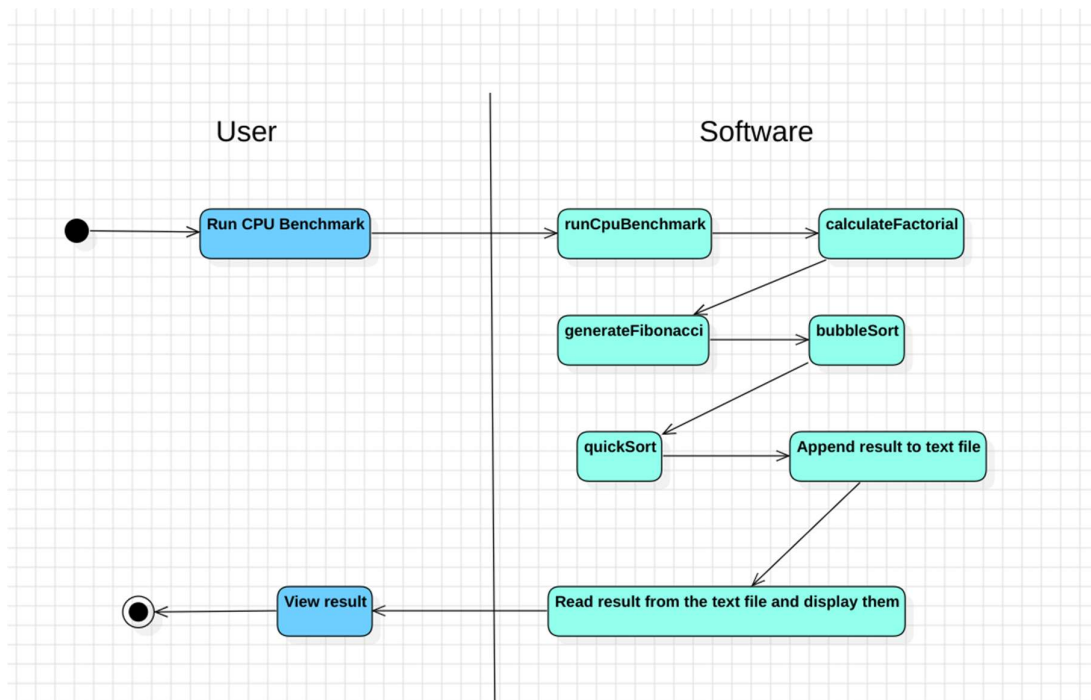
- **Type:** Definirea tipurilor de tema
 - **Theme:** Contine logica pentru aplicarea unui stil general al aplicatiei, integrand diferite optiuni de personalizare
 - **Color:** Asigura setarile de culoare pentru a personaliza aspectul aplicatiei in functie de tema selectata
3. **Benchmarks:** Acest pachet include modulele esentiale pentru testele de performanta, fiecare realizand un tip specific de benchmark:
- ✓ **CpuBenchmark:** responsabil pentru evaluarea performantei procesorului (CPU) dispozitivului mobil
 - ✓ **GpuBenchmark:** efectueaza teste de performanta grafica, analizand capabilitatile GPU-ului
 - ✓ **MemoryBenchmark:** masoara performantele de citire/scriere pentru memoria RAM
 - ✓ **HardwareInfo:** colecteaza informatii despre configuratia hardware a dispozitivului, oferind o vedere de ansamblu asupra specificatiilor
4. **Main:** Acest pachet cuprinde clasa **MainActivity**, componenta principala a proiectului care faciliteaza interactiunile cu utilizatorul si integreaza functionalitatile din celelalte pachete (Benchmarks si UI Theme), coordonand initierea testelor de benchmark si aplicarea temei UI.

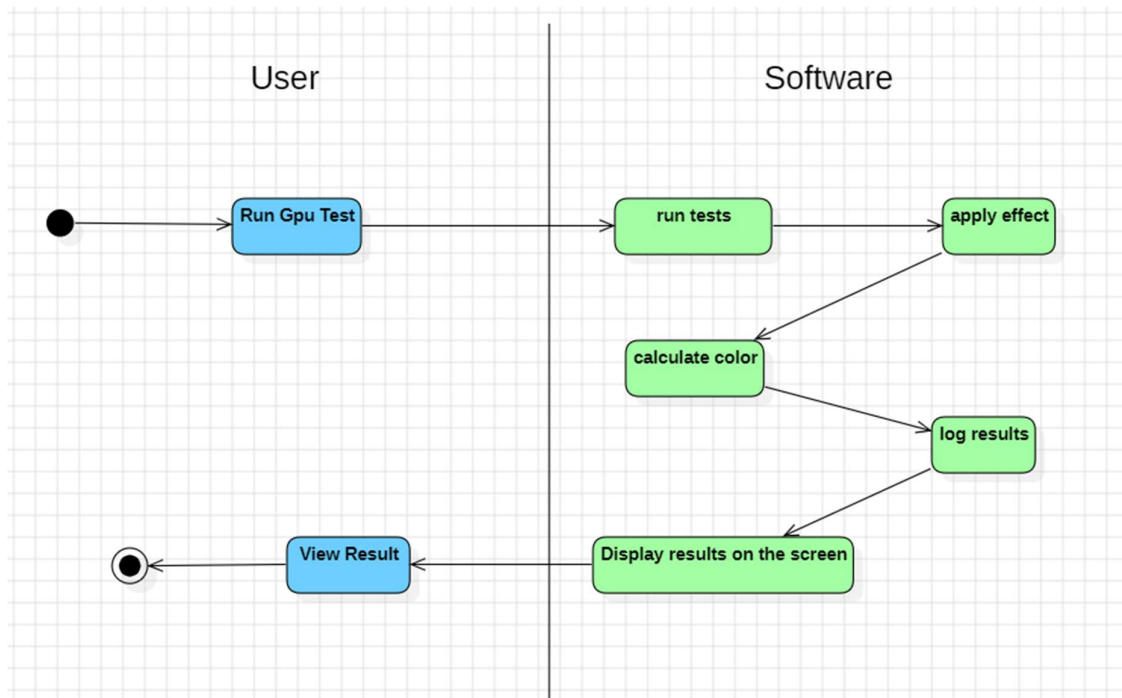
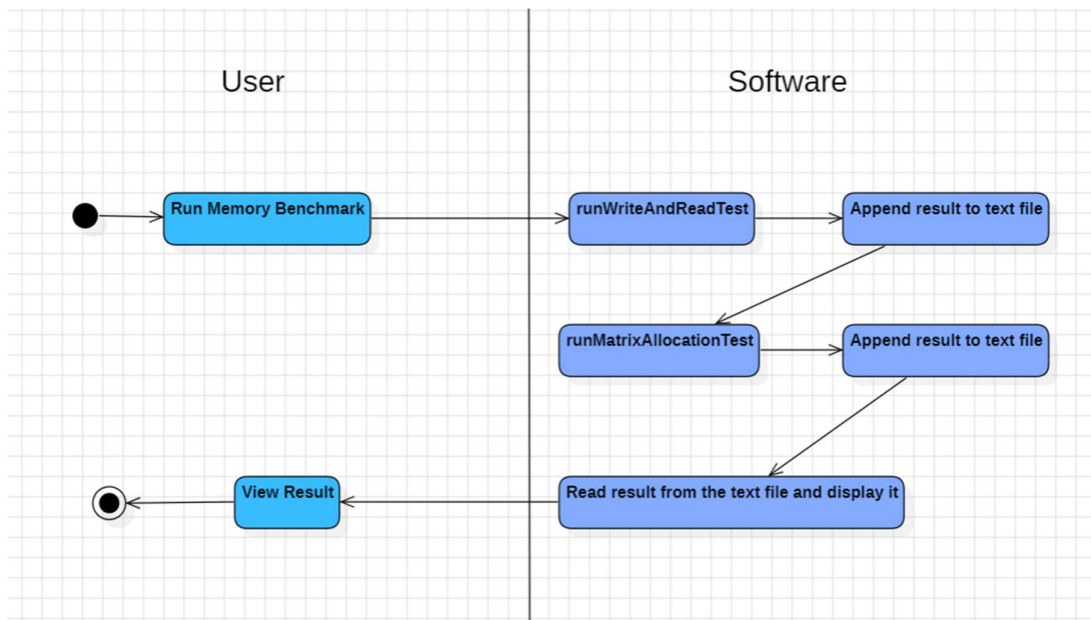
4.1.2 Diagrame de Activitate

Rolul unei diagrame de activitati este de a ilustra fluxul de actiuni si secventa operatiunilor intr-un proces sau sistem. Acestea descriu cum se succed activitatile, evidentiind interactiunile dintre elementele sistemului (e.g. utilizatori si componentele software). Elementele cheie intr-o diagrama de activitati includ actiunile propriu-zise (reprezentate de blocuri dreptunghiulare), sageti de tranzitie (care indica fluxul progresului) si puncte de inceput si de sfarsit pentru a delimita inceputul si finalul procesului.

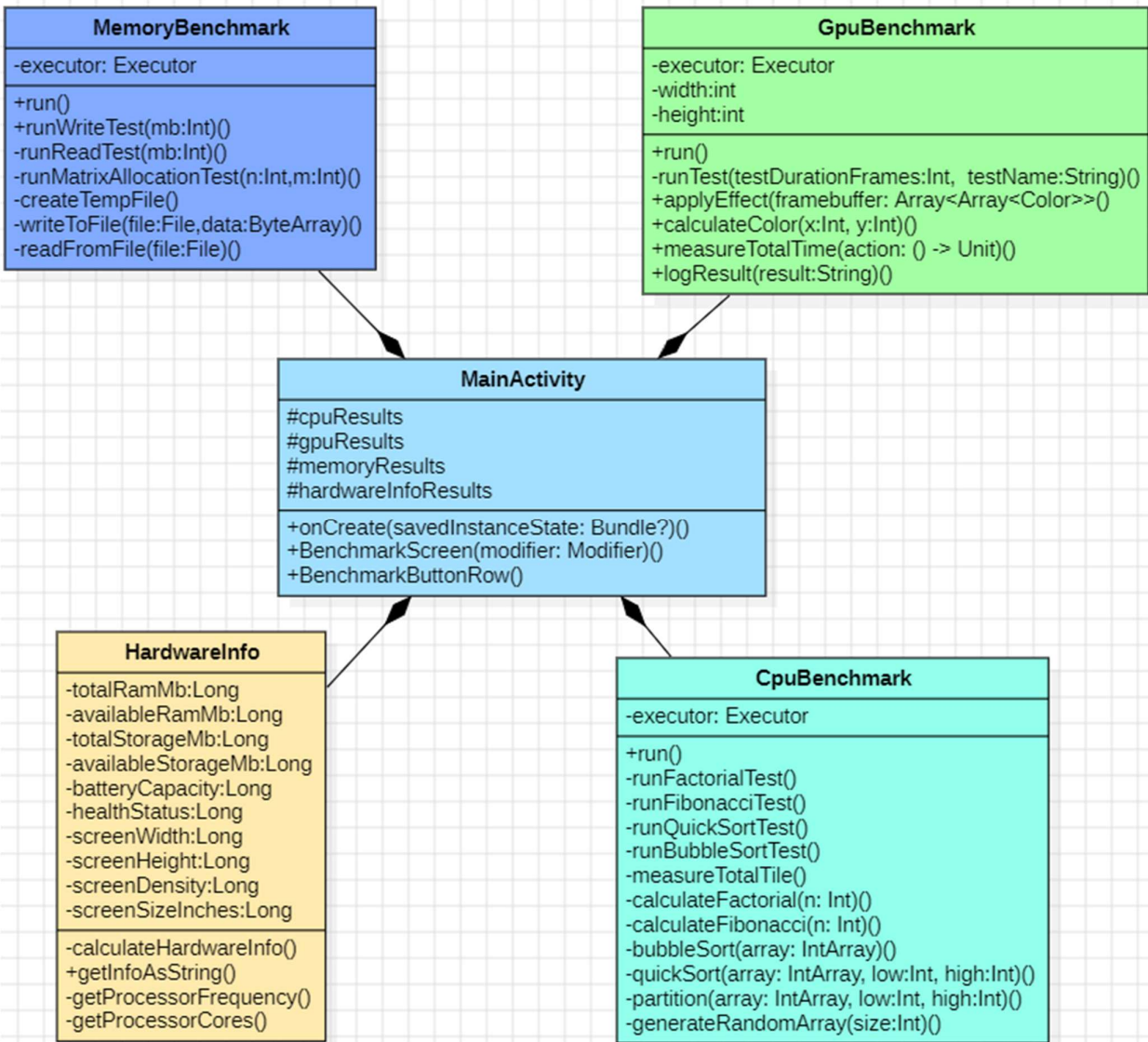
In general o astfel de diagrama are 2 componente (parti):

1. **User:** secventa de actiuni ale utilizatorului care reprezinta startul procesului si urmeaza sa fie transmise mai departe pentru o procesare din partea software-ului
2. **Software:** aici au loc operatiunile complexe realizate de calculator prin intermediul algoritmilor. Acestea au scopul de a duce la indeplinire comenzile user-ului si de furniza rezultate conforme cerintelor acestuia





4.1.3 Diagrama de Clase



5.Implementare

5.1 Structura pachetelor proiectului

Proiectul este organizat in 3 pachete principale: **Benchmarks**, **Main** si **UI.theme**.

Aceasta structurare modulara asigura o separatie clara a responsabilitatilor, ceea ce faciliteaza intretinerea, extinderea si intelegerea proiectului.

5.1.1 Pachetul Benchmarks

Acest pachet reprezinta nucleul logicii aplicatiei si include clasele care implementeaza testele de performanta pentru componentele hardware. In cadrul acestui pachet se afla urmatoarele clase:

- **CpuBenchmark**: implementarea testelor de performanta pentru procesor. Aceasta clasa contine metode pentru a evalua capabilitatile CPU, inclusiv executia de calcule complexe, cum ar fi factorial, generarea de numere Fibonacci si algoritmi de sortare.
- **GpuBenchmark**: construita pentru a testa performanta grafica a dispozitivului, aceasta clasa este responsabila pentru rularea unor sarcini intensive pe GPU, cum ar fi randarea de grafica.
- **HardwareInfo**: ofera informatii detaliate despre hardware-ul dispozitivului android, incluzand detalii despre memoria RAM, spatiul de stocare al telefonului, baterie si display. Este esentiala pentru a completa profilul hardware inainte sau dupa rularea testelor
- **MemoryBenchmark**: contine metode pentru a testa performanta memoriei, folosind teste de scriere si citire de fisiere mari din memorie, precum si alocarea unor structuri de date complexe in memorie.

5.1.2 Pachetul Main

Acest pachet contine principala clasa a aplicatiei

- **MainActivity**: serveste drept punct de intrare in aplicatie. MainActivity gestioneaza interactiunea cu utilizatorul si initiaza conexiunea testelor din pachetul **Benchmarks**. Aceasta clasa joaca un rol central in executia testelor din pachetul **Benchmarks**.

5.1.3 Pachetul UI.Theme

Acest pachet definește aspectul vizual al aplicației, inclusiv culori pentru fundaluri, text, butoane etc.

- **Color.kt:** definește pacheta de culori utilizată în aplicație (culori pentru text, fundaluri, butoane etc.)
- **Theme.kt:** configurează tema globală a aplicației, oferind suport pentru moduri Light și Dark. De asemenea, gestionează setările pentru tipografie și culori.
- **Type.kt:** conține definițiile pentru stilurile de text utilizate în aplicație, cum ar fi dimensiuni, fonturi pentru diferite titluri, paragrafe și alte elemente.

5.1.5 Sinergia dintre pachete

- Pachetul **Benchmarks** realizează procesarea detaliată a testelor hardware executate
- Pachetul **Main** orchestrează logica aplicației și leagă logica de interfața grafică
- Pachetul **UI.Theme** oferă un design plăcut și funcțional, asigurând utilizatorului o experiență clară și estetică.

5.2 Structura clasei MainActivity

Clasa **MainActivity** reprezintă punctul de intrare în aplicația Mobile Benchmark App. Aceasta utilizează biblioteca **Jetpack Compose** pentru a crea o interfață utilizator declarativă și include logica necesară pentru a coordona interacțiunile utilizatorului cu diferite benchmark-uri (CPU, GPU, memorie) și pentru afișarea informațiilor hardware.

5.2.1 Structura generală

Clasa extinde **ComponentActivity**, o clasă specifică din **Jetpack Compose** care facilitează utilizarea UI-urilor declarative în aplicațiile Android. Activitatea principală inițializează tema aplicației și afișează componenta principală de UI denumită **BenchmarkScreen**.

Metoda **onCreate** este metoda principală care se execută la crearea activității. În cadrul ei:

- ✓ Se setează conținutul UI folosind metoda **setContent**, specifică pentru **Jetpack Compose**

- ✓ Tema aplicatiei este definita prin **MobileBenchmarkAppTheme**, asigurand un design consistent.
- ✓ Se afiseaza ecranul principal prin componenta **BenchmarkScreen**, la care se adauga padding-ul definit de **Scaffold**.

5.2.2 Componenta BenchmarkScreen

Aceasta este componenta principala care defineste structura si comportamentul UI-ului aplicatiei. Este marcata cu anotarea **@Composable** ceea ce indica faptul ca este o functie declarativa utilizata pentru a construi interfata utilizatorului.

Variable si initializari

- ✓ **Context**: obtine contextul aplicatiei folosind **LocalContext.current**. Contextul este esential pentru initializarea claselor de benchmark si obtinerea resurselor locale.
- ✓ **State variables**
 - **cpuResults**, **gpuResults**, **memoryResults**, **hardwareInfoResults**: Variabile mutabile (**mutableStateOf**) utilizate pentru a stoca rezultatele fiecarui benchmark si informatiile hardware. Acestea sunt reactivate automat de Compose la schimbarea valorilor lor.

Initializarea obiectelor

- **CpuBenchmark**: initializeaza testul pentru procesor, rezultatele fiind stocate in **cpuResults**
- **MemoryBenchmark**: Initializeaza testul pentru memorie, rezultatele fiind salvate in **memoryResults**
- **GpuBenchmark**: initializeaza testul pentru GPU, cu salvarea rezultatelor in **gpuResults**
- **HardwareInfo**: Initializeaza clasa pentru obtinerea informatiilor hardware, valorile fiind utilizate la afisarea detaliilor dispozitivului

Structura Interfetei Grafice pentru Utilizator

GUI-ul principal este construit in mai multe sectiuni folosind componente declarative din Compose:

- **Titlul Aplicatiei:** Afiseaza titlul aplicatiei in partea de sus a ecranului, folosind un stil de titlu mediu predefinit.
- **Grupuri de butoane(BenchmarkButtonRow):** componente personalizate care organizeaza doua butoane pe un rand pentru a lansa diferite teste:
 - **Primul rand:**
 - Butonul „Run CPU Benchmark” lanseaza testul pentru procesor
 - Butonul „Run GPU Benchmark” lanseaza testul grafic
 - **Al doilea rand:**
 - Butonul „Run Memory Benchmark” lanseaza testul pentru memorie
 - Butonul „Show Hardware Info” afiseaza detaliile hardware ale dispozitivului
- **Lista de rezultate (Lazy Column):** Afiseaza rezultatele fiecarui test intr-un format de tip lista, unde fiecare sectiune este reprezentata printr-un text:
 - **Rezultatele pentru CPU**
 - **Rezultatele pentru GPU**
 - **Rezultatele pentru memorie**
 - **Informatiile hardware**

5.2.3 Componenta BenchmarkButtonRow

Aceasta functie composable este utilizata pentru a afisa doua butoane pe acelasi rand. Este reutilizata pentru fiecare pereche de actiuni (benchmark-uri sau afisarea informatiilor hardware)

Parametri:

- **buttonText1, buttonText2:** Textul afisat pe fiecare buton
- **onClick1, onClick2:** Functii callback care definesc actiunile ce trebuie efectuate la apasarea fiecarui buton

Structura UI:

- **Row:** componenta ce genereaza butoanele pe un rand cu spatiere uniforma
- **Button:** Componente individuale e tip buton, fiecare configurata cu textul si functia sa de click

Rolul metodei Scaffold

În cadrul metodei `setContent`, este utilizată componenta `Scaffold` pentru a gestiona structura generală a aplicației, incluzând padding-ul și dimensiunea întregului ecran. Aceasta este o practică recomandată în Jetpack Compose pentru a asigura un layout adaptabil și organizat.

5.2.4 Fluxul de funcționare

1. **Interacțiunea utilizatorului:**
 - a. Utilizatorul apasă un buton pentru a inițializa un test specific sau pentru a afișa informațiile hardware
 - b. Callback-ul asociat butonului actualizează variabila de stare corespunzătoare (`cpuResults`, `gpuResults`, etc.)
2. **Actualizare UI-ului:** Jetpack Compose detectează modificările de stare și reface automat componentele UI relevante pentru a reflecta noile valori
3. **Afișarea rezultatelor:** rezultatele sunt afișate în secțiunile din `LazyColumn`, organizate sub formă de text

5.2.5 Calcularea Scorului final

Funcționalitatea de calculare a scorului oferă utilizatorului o evaluare unificată a performanței dispozitivului, exprimată ca un procent.

Fluxul de calcul al scorului:

1. **Declansarea calculului:** funcția `calculateAndSetScore()` este responsabilă de calcularea scorului. Este apelată doar atunci când cele patru teste sunt finalizate, statusul fiecărui test fiind marcat printr-o variabilă booleană.
2. **Algoritmul propriu-zis:** funcția preia rezultatele testelor și le analizează pentru a determina un scor final. Algoritmul asigură o pondere adecvată a rezultatelor testelor în calculul scorului.
3. **Actualizare UI:** după calculul scorului, acesta este trimis printr-o funcție de callback `onScoreCalculated()` pentru a fi afișată pe ecran, astfel putând fi interpretată de utilizator.

5.3 Structura clasei CpuBenchmark

Clasa CpuBenchmark este responsabila pentru testarea performantei procesorului (CPU) al dispozitivului Android prin rularea mai multor algoritmi intensivi din punct de vedere computational.

Aceasta clasa include patru teste distincte: factorial, Fibonacci, sortare prin metoda Bubble Sort si sortare rapida (Quick Sort). Rezultatele testelor sunt salvate intr-un fisier si afisate utilizatorului printr-o functie callback.

Atributele Clasei

- 1) **context:Context** – reprezinta contextul aplicatiei Android, utilizat pentru accesarea sistemului de fisiere pentru salvarea rezultatelor testului
- 2) **onResult: (String) -> Unit** – o functie callback folosita pentru a returna rezultatele testelor catre interfata grafica sau alta componenta a aplicatiei
- 3) **executor:Executors** – un executor service configurat cu ajutorul unui thread pool de dimensiune 4 care permite rularea simultana a celor patru teste de CPU. Astfel, fiecare test ruleaza intr-un fir de executie separat pentru a masura performanta fara a bloca interfata grafica
- 4) **resultsFile:File** – fisierul in care sunt salvate rezultatele testelor CPU. Acesta este localizat in directorul aplicatiei (**context.filesDir**) si poarta numele de **cpu_results.txt**

Metode Publice:

- **fun run():** Initializeaza rularea celor patru teste CPU folosind executorul. Fiecare test este rulat pe un thread diferit pentru a imbunatati eficienta si a permite rularea paralela

Metode Private:

1. **private fun runFactorialTest():** testeaza performanta CPU calculand factorialul unui numar foarte mare (100.000) de 10 ori. Timpul total pentru cele 10 calcule este masurat si inregistrat folosind **measureTotalTime**. Rezultatul este scris in fisier prin metoda **logResult**
2. **private fun runFibonacciTest():** testeaza performanta CPU, generand seria Fibonacci pana la al 100.000 termen, de 10 ori. Rezultatul este logat.

3. **private fun runBubbleSortTest():** evalueaza performanta CPU sortand un vector de dimensiune mare (25.000) folosind algoritmul Bubble Sort. Sortarea este repetata de 10 ori, timpul total fiind masurat.
4. **private fun runQuickSortTest():** evalueaza performanta procesorului sortant un vector de dimensiune mare (25.000), folosind algoritmul Quick Sort (un algoritm mai eficient decat Bubble Sort). Similar cu celelalte teste, sortarea e repetata de 10 ori, timpul total fiind masurat.
5. **private inline fun measureTotalTime(action: () -> Unit):Long** - o functie ajutatoare care masoara timpul total necesar pentru executarea unei actiuni, utilizand **measureTimeMillis**. Parametrul action este o functie lambda care reprezinta operatiunea de testat
6. **private fun calculateFactorial(n: Int):Long** – calculeaza factorialul unui numar n folosind o bucla iterativa. Complexitate: $O(n)$
7. **private fun generateFibonacci(n: Int):Long** – genereaza primii n termeni din sirul lui Fibonacci folosind o abordare itelativa. Complexitate: $O(n)$
8. **private fun bubbleSort(array: IntArray): IntArray** – Implementeaza algoritmul Bubble Sort pentru sortarea unui vector. Complexitate: $O(n)$.
9. **private fun quickSort(array: IntArray, low:Int, high:Int):IntArray** – Implementeaza algoritmul de sortare rapida pentru a partitiona vectorul in doi subvectori, sortand-ui recursiv. Complexitate: $O(n \log n)$ [in cazul average], $O(n^2)$ [in worst case scenario]
10. **private fun partition(array: IntArray, low:Int, high:Int):Int** – o metoda auxiliara pentru quick sort care determina pozitia pivotului si rearanjeaza vectorul astfel incat elementele mai mici sa fie pe partea stanga, iar cele mai mari pe partea dreapta.
11. **private fun IntArray.swap(i:Int, j:Int)** – o metoda auxiliara pentru a interschimba doua elemente de tip IntArray
12. **private fun generateRandomArray(size: Int):IntArray** – creeaza si returneaza un vector de dimensiune size cu valori intregi random din intervalul [0,99999]
13. **private fun logResult(result:String)** – salveaza rezultatul unui test de benchmark intr-un fisier (cpu_results.txt)

5.4 Structura clasei **MemoryBenchmark**

Clasa **MemoryBenchmark** are responsabilitatea de a evalua performanta memoriei RAM a dispozitivului, folosind operatii de citire din memorie si scriere in memorie, precum si operatii de alocare de memorie. Aceasta clasa executa trei teste principale.

Atribute

1. **context:Context**: contextul aplicatiei Android. Este utilizat pentru accesarea sistemului de fisiere, pentru crearea fisierele temporare si salvarea rezultatelor testelor.
2. **onResult: (String) -> Unit** – O functie callback utilizata pentru a returna rezultatele testelor catre interfata grafica sau alta componenta.
3. **executor:Executors** – Un executor service cu trei fire de executie distincte. Fiecare test de memorie ruleaza pe un thread separat ceea ce permite executia lor simultana si mai eficienta
4. **resultsFile:File** – Fisierul in care sunt salvate rezultatele testelor de memorie. Acesta e localizat in directorul aplicatiei, iar numele fisierului este **memory_results.txt**

Metode publice

- **fun run()** – Declanseaza rularea celor trei teste de memorie in paralel: testul de scriere, testul de citire si testul de alocare in memorie.

Metode private

1. **private fun runWriteTest(mb:Int)**: testeaza viteza de scriere a unui fisier temporar de dimensiune mb(in megabytes) pe disc. Pasii acestei metode sunt:
 - a. creeaza un fisier temporar in memorie
 - b. genereaza un vector mare de tip **ByteArray** cu valori aleatoare
 - c. scrie acest vector in fisier si masoara timpul necesar folosind **measureTimeMillis**
 - d. logheaza rezultatul si sterge fisierul temporar
2. **private fun runReadTest(mb:Int)**: calculeaza viteza de citire a unui fisier temporar de dimensiune mb(in megabytes) de pe disc. Pasi:
 - a. creeaza un fisier temporar
 - b. genereaza si scrie un vector de dimensiuni mari de tip **ByteArray** in fisier
 - c. citeste continutul fisierului si masoara timpul necesar
 - d. logheaza rezultatul si sterge fisierul temporar

3. **private fun runMatrixAllocationTest(n:Int, m:Int)** – evalueaza viteza de alocare si initializare a unor matrice de dimensiuni mari. Pasii sunt:
- Se aloca **n** matrice, fiecare de dimensiune **m*m**, fiecare element fiind un numar aleatoriu.
 - Masoara timpul total necesar pentru alocare folosind **measureTimeMillis**
 - Logheaza rezultatul

Metode auxiliare

1. **private fun createTempFile():File** – creaza un fisier temporar cu extensia **.tmp**
Fisierul este setat sa fie sters automat cand aplicatia este inchisa (**deleteOnExit**)
2. **private fun writeToFile(file:File, data:ByteArray)** – scrie un vector de octeti intr-un fisier folosind un obiect de tip **RandomAccessFile**
3. **private fun readFromFile(file:File):ByteArray** – citeste continutul unui fisier si returneaza un vector de octeti
4. **private fun logResult(result:String)** – salveaza rezultatul unui test. Pe de-o parte, il trimite functiei callback **onResult** pentru afisare imediata pe ecran. Pe de alta parte, il scrie in fisierul **memory_results.txt** pentru stocare permanenta.

5.5 Structura clasei HardwareInfo

Clasa HardwareInfo ofera o metoda detaliata de colectare a informatiilor hardware si de sistem pentru un dispozitiv Android. Informatiile colectate includ date despre memorie, stocare, display, baterie si procesor.

Atribute

✓ **Memorie si stocare:**

- totalRamMb – memoria RAM totala a dispozitivului
- availableRamMb - memoria RAM disponibila (libera) in momentul testarii
- totalStorageMb - stocarea interna totala
- availableStorageMb – stocarea interna disponibila in momentul testarii

✓ **Baterie:**

- batteryCapacity – capacitatea actuala a bateriei, exprimata in procente
- healthStatus – starea de sanatate a bateriei (ex. „Good”, „Overheat”)

✓ **Display:**

- screenWidth – latimea ecranului in pixeli
- screenHeight – inaltimea ecranului in pixeli
- screenDensity – densitatea pixelilor (dpi)
- screenSizeInches – dimensiunea aproximativa a ecranului in inchi

✓ **Fisier pentru rezultate:**

- resultsFile – fisier local pentru stocarea rezultatelor hardware (**hardware_results.txt**), localizat in directorul aplicatiei

Metode publice

1. **init** – constructorul ce initializeaza datele hardware apeland metoda privata `calculateHardwareInfo()`
2. **fun getInfoAsString(): String** – returneaza toate informatiile hardware sub forma unui sir
 - a. informatiile sunt structurate in categorii: detalii despre dispozitiv, memorie, stocare, baterie si afisaj
 - b. rezultatul este salvat in fisierul `hardware_results.txt`

Metode private

1. **private fun calculateHardwareInfo()** – colectează datele hardware și le salvează în atributele clasei
 - a. **Memorie:** folosește **ActivityManager**.memoryInfo pentru a obține memoria totală și cea disponibilă
 - b. **Stocare:** folosește **StatFs** pentru a determina spațiul total și cel disponibil din stocarea internă
 - c. **Baterie:** obține procentul de încărcare și starea de sănătate folosind **BatteryManager** și **IntentFilter**
 - d. **Display:** calculează dimensiunea ecranului în inch, bazându-se pe dimensiunea pixelilor (**pixelMetrics**)
2. **private fun getProcessorFrequency(): Int** - determină frecvența procesorului în MHz
 - a. Accesează fișierul sistemului
`/sys/devices/system/cpu/cpu0/cpufreq/scaling_max_freq`
 - b. În caz de eroare, returnează 0
3. **Private fun getProcessorCores():Int** – returnează numărul total de nuclee disponibile pe procesor, folosind `Runtime.getRuntime().availableProcessors()`
4. **Private fun logResult(result:String)** – salvează rezultatul hardware într-un fișier text (`hardware_results.txt`) folosind un `FileWriter`

Fluxul execuției

1. Inițializarea informațiilor hardware
 - a. La instanțierea clasei, metoda `calculateHardwareInfo()` colectează toate datele necesare despre dispozitiv
 - b. Informațiile hardware sunt stocate în atributele clasei
2. Obținerea informațiilor hardware
 - a. Metoda `getInfoAsString()` formatează toate informațiile colectate într-un șir structurat și le returnează
 - b. Rezultatul este, de asemenea, salvat local într-un fișier text
3. Procesor
 - a. Informațiile despre procesor (frecvența maximă și numărul de nuclee disponibile) sunt de asemenea calculate
4. Persistența rezultatelor
 - a. Toate rezultatele sunt salvate într-un fișier text pentru referințe ulterioare

5.5 Structura clasei GpuBenchmark

Clasa GpuBenchmark este responsabila pentru evaluarea performantei GPU-ului dispozitivului Android prin simularea unei serii de cadre grafice si aplicarea unor efecte vizuale asupra framebuffer-ului. Testele sunt executate in paralel (tehnica multi-threading) pentru a utiliza eficient resursele dispozitivului.

Atribute

1. **Context:Context** – contextul aplicatiei Android. Este utilizat pentru accesarea sistemului de fisiere si pentru salvarea rezultatelor testelor GPU.
2. **onResult: (String) -> Unit** – o functie callback utilizata pentru afisarea sau transmiterea rezultatelor catre interfata grafica sau alte componente.
3. **Executor:Executors** – un pool de fiere de executie cu 4 thread-uri. E folosit pentru rularea in paralel a testelor GPU
4. **resultsFile:File** – fisierul in care sunt salvate rezultatele testelor GPU. Fisierul poarta denumirea de **gpu_results.txt** si este localizat in directorul aplicatiei.
5. **Width: Int** – latimea frame buffer-ului utilizat pentru simularea graficii. Valoare implicita 1920.
6. **Height: Int** – inaltimea frame buffer-ului utilizat pentru simularea graficii. Valoare implicita 1080
7. **Color** – Structura RGBA pentru reprezentarea culorii fiecarui pixel. Atribute:
 - a. **R: Byte** – componenta rosie
 - b. **G: Byte** – componenta galbena
 - c. **B: Byte** – componenta albastra
 - d. **A:Byte** – componenta alpha (implicit 255)

Metode publice:

1. Fun run() – porneste rularea testelor GPU in paralel. Testele ruleaza pentru 5, 10, 15, 20 cadre, fiecare test fiind executat pe un thread separat din pool-ul **executor**.

Metode private:

1. **Private fun runTest(testDurationFrames: Int, testName: String)** – ruleaza un test GPU pentru un numar specificat de cadre. Pasi:
 - a. Creeaza un **framebuffer** bidimensional (o matrice de pixeli)
 - b. Aplica efecte grafice pe fiecare cadru, simuland procesarea grafica
 - c. Masoara timpul total necesar folosint **measureTimeMillis**
 - d. Calculeaza timpul mediu pentru procesarea fiecarui cadru si logheaza rezultatul

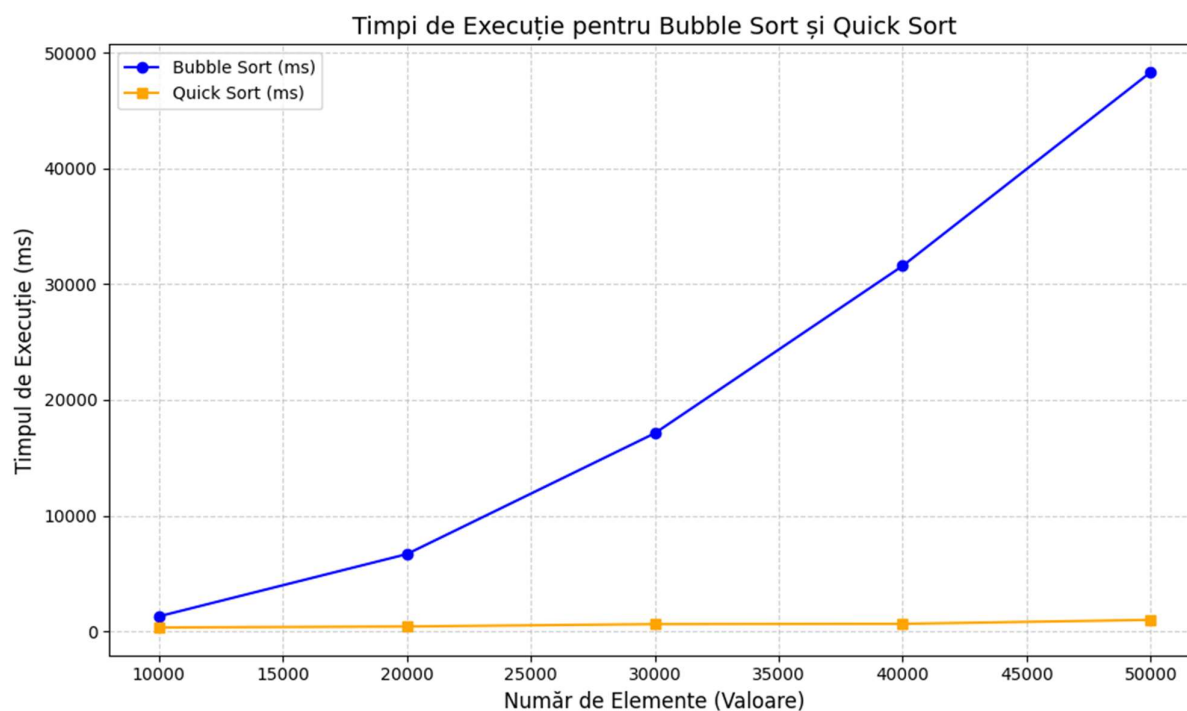
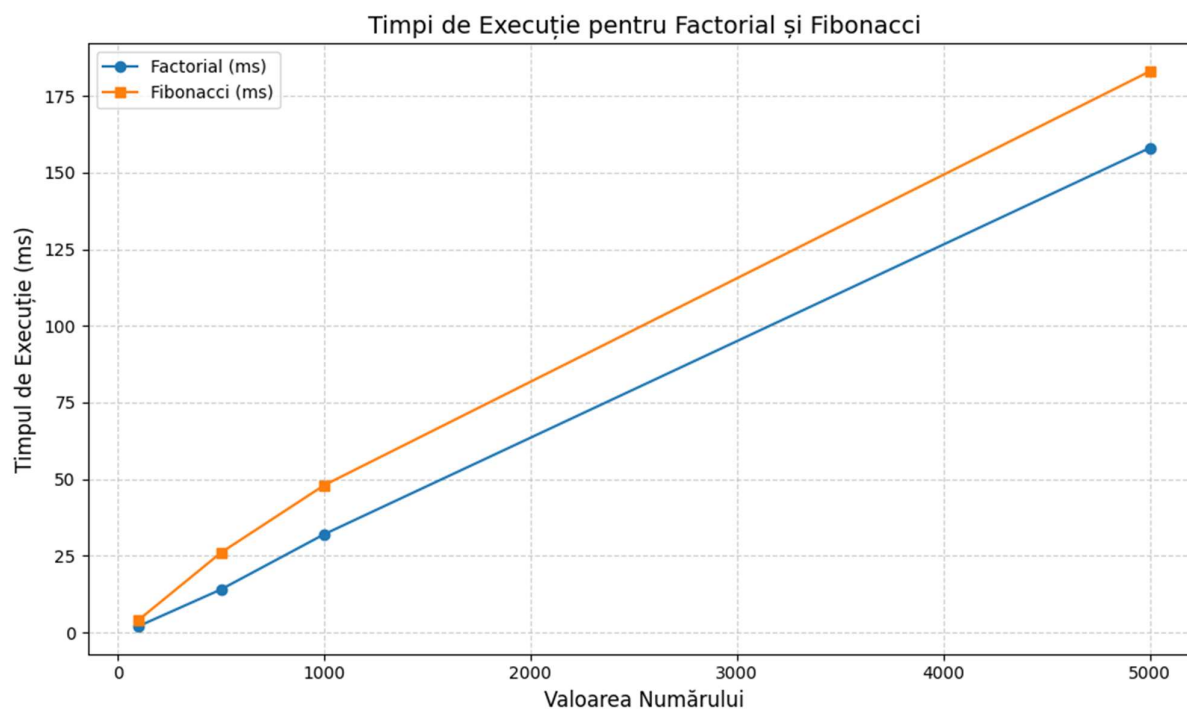
2. **Private fun applyEffect(framebuffer: Array<Array<Color>>)** – aplica un test grafic pe fiecare pixel al framebuffer-ului. Pasi:
 - a. Itereaza prin fiecare pixel din matrice
 - b. Calculeaza o culoare noua pentru fiecare pixel folosind functia **calculateColor**
3. **Private fun calculateColor(x:Int, y:Int): Color** – genereaza o culoare pentru un pixel pe baza pozitiei sale (x,y). Pasi:
 - a. $r = (x * y) \% 256$
 - b. $g = (x * y) \% 256$
 - c. b = valoare aleatoare intre 0 si 255
4. **Private inline fun measureTotalTime(action: ()-> Unit): Long** – masoara timpul total necesar pentru o actiune specificata.
5. **Private fun logResults(result:String)** – salveaza rezultatul unui test in doua moduri:
 - a. Trimite rezultatul functiei callback **onResult** pentru afisare pe ecran
 - b. Scrie rezultatul in fisierul **gpu_results.txt** pentru stocare permanenta

Fluxul executiei

1. Rularea Testelor – metoda run() initializeaza executia a patru teste GPU in paralel:
 - a. Test 1: 5 cadre
 - b. Test 2: 10 cadre
 - c. Test 3: 15 cadre
 - d. Test 4: 20 cadre
2. Calcularea culorilor – fiecare pixel din framebuffer va primi o culoare calculata pe baza pozitiei (x,y) si a unor valori aleatoare, simuland procesele grafice complexe.

6.Testare si Validare

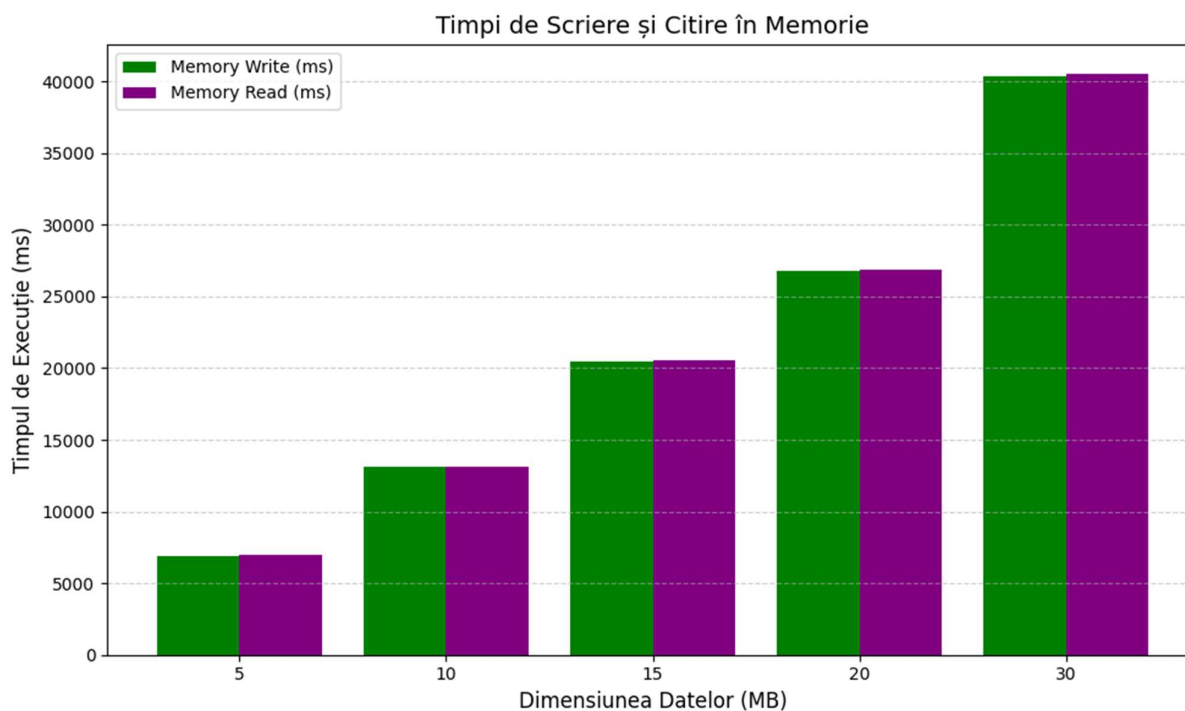
1. Testare si validare CPU

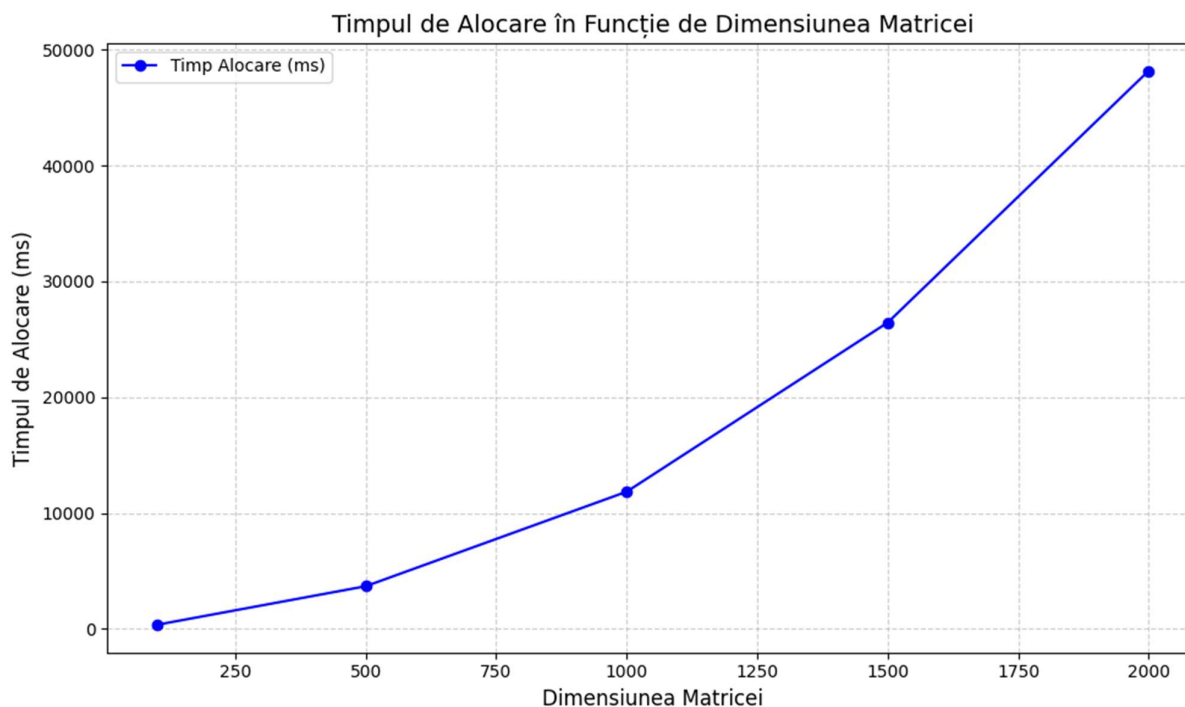


Interpretare generala:

- **Factorial Test:** valorile (reprezentate de linia albastra) sunt mai mici si constau in calcule repetate pentru factorialul numarului 1000. Valorile mai scazute sugereaza ca acest test necesita ceva mai putin timp decat testul Fibonacci
- **Fibonacci Test:** valorile (reprezentate de linia portocalie) sunt in mod constant mai mari decat cele pentru testul Factorial, indicand faptul ca generarea sirului Fibonacci pana la al 1000-lea numar este o sarcina mai solicitanta pentru CPU si dureaza mai mult.
- **Bubble Sort (barele albastre):** timpul de executie creste semnificativ odata cu numarul de elemente din sir. Acest lucru evidentiaza ineficienta algoritmului Bubble Sort pentru seturi mari de date
- **Quick Sort (barele portocalii):** Timpul de executie ramane relativ scazut si constant, chiar si pe masura ce numarul de elemente creste. Acest lucru releva eficienta algoritmului Quick Sort in comparatie cu Bubble Sort, in special pentru seturi mari de date.

2. Testare si validare Memory





Interpretare Read and Write Test

Acest grafic compara performanta operatiilor de citire din memorie si scriere in memorie pentru diferite puncte de date (5,10,15,20 si 30 MB)

- **Scrierea memoriei (barele albastre):** timpul necesar pentru scrierea memoriei este putin mai mic decat cel pentru citire, in toate punctele de date
- **Citirea memoriei (la barele portocalii):** necesita putin mai mult timp in comparatie cu scrierea, diferentele se diminueaza pe masura ce volumul de date creste

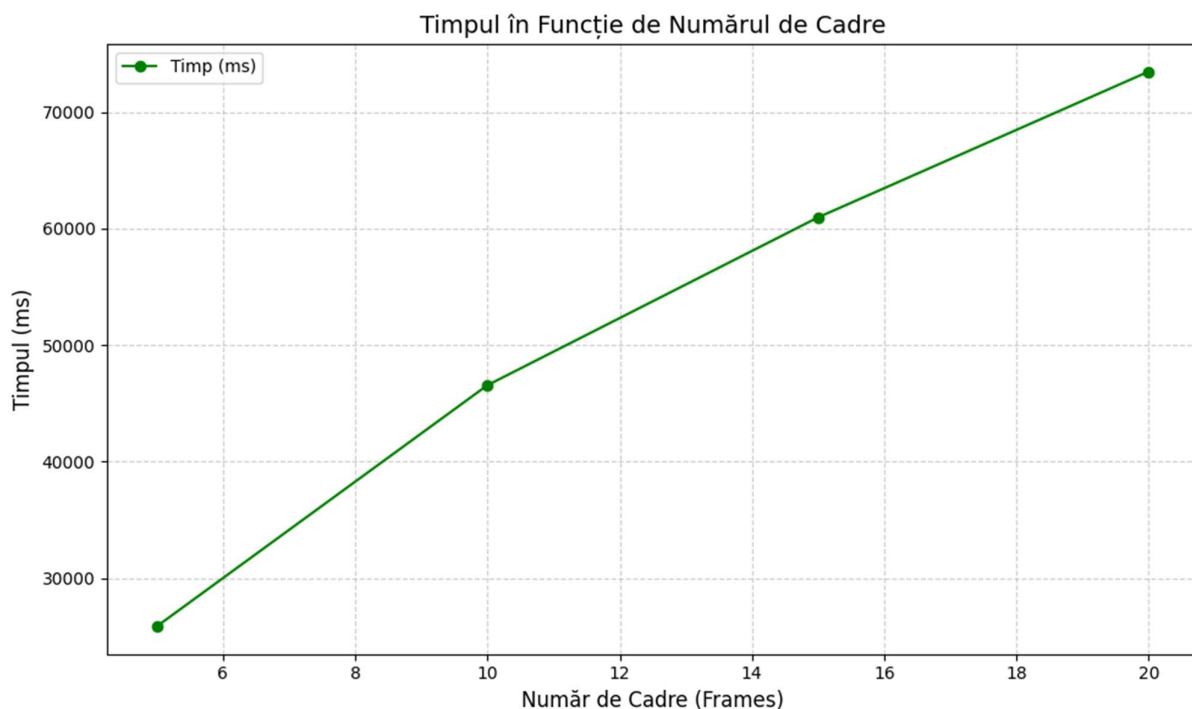
Intepretare Generala Allocation Time

Acest grafic ilustreaza timpul de alocare in memorie relativ la dimensiunile matricei

- Timpul de alocare creste exponential pe masura ce dimensiunea matricei creste
- La 500x500 timpul de alocare este aprox. 4000 ms (4 secunde)
- La 1000x1000 timpul de alocare este aprox. 12000 ms (12 secunde)
- La 2000x2000 timpul de alocare este aprox. 48000 ms (48 secunde)

Aceste date sugereaza ca alocarea in memorie a unor matrice mari poate fi foarte costisitoare in termeni de timp si resurse, subliniind importanta optimizarii acestor operatii in aplicatiile care necesita manipularea intensa a datelor.

3. Testare si validare GPU



Interpretare generala si observatii

Graficul furnizeaza informatii cu privire la performanta GPU-ului in functie de numarul de cadre procesate. Pe axa X se afla numarul de cadre, iar pe axa Y se afla timpul total necesar pentru procesarea acestor cadre, reprezentat in milisecunde (ms).

- ✓ Trend ascendent clar: pe masura ce numarul de cadre procesate creste, timpul total de procesare creste si el liniar. Acest lucru indica faptul ca GPU -ul necesita mai mult timp pentru a randa un numar mai mare de cadre.
- ✓ Puncte de date specifice:
 - La 5 cadre, timpul de procesare este aprox. 25000 ms (25 secunde)
 - La 10 cadre, timpul de procesare este aprox. 46000 ms (46 secunde)
 - La 15 cadre, timpul de procesare este aprox. 60000 ms (60 secunde)
 - La 20 cadre, timpul de procesare ajunge la 75000 ms (75 secunde)
- ✓ Performanta GPU: graficul sugereaza ca performanta GPU scade pe masura ce numarul de cadre creste, datorita cresterii cerintelor de procesare
- ✓ Scalabilitate: GPU-ul poate gestiona un numar mai mare de cadre, dar timpul necesar creste semnificativ, ceea ce poate afecta performanta aplicatiilor grafice intensive

7. Concluzii

7.1 Ce am invatat din redactarea proiectului

Dezvoltarea proiectului „*Android Benchmark App*” a oferit o intelegere solida asupra procesului de creare a unei aplicatii de benchmarking pentru platforma Android, utilizand tehnologii moderne precum Jetpack Compose. Printre cele mai importante lectii invatate se numara:

- ✓ **Gestionarea unei interfete declarative:** utilizarea Jetpack Compose a simplificat semnificativ crearea si actualizarea interfeței utilizatorului
- ✓ **Organizarea logicii de testare:** crearea claselor pentru fiecare tip de test a oferit o modularitate excelenta, facilitand extinderea sau ajustarea ulterioara a codului
- ✓ **Integrarea si sincronizarea datelor:** am invatat coordonarea testelor si gestionarea acestora intr-un mod coerent, folosind tehnica de multi-threading.
- ✓ **Lucrul cu starea aplicatiei:** utilizarea variabilelor mutabile in Compose ne-a oferit un control detaliat asupra reactualizarii interfeței grafice, fara a necesita interventii suplimentare pentru sincronizare manuala.

7.2 Posibilitati de dezvoltare ulterioara

1. **Adaugarea unor noi benchmark-uri:** implementarea testelor pentru viteza rețelei sau viteza memoriei de stocare (cum ar fi carduri SD)
2. **Vizualizari grafice mai avansate:** pentru o interpretare mai detaliata si mai avansata a rezultatelor obtinute in urma rularii testelor
3. **Optimizarea algoritmilor de benchmarking**
4. **Integrarea unui istoric al rezultatelor:** salvarea rezultatelor intr-o baza de date locala pentru a permite utilizatorului sa compare performantele dispozitivului in timp sau in diferite conditii de utilizare
5. **Personalizarea si partajarea rezultatelor:** permisiunea utilizatorilor de a personaliza testele pe care doresc sa le ruleze si integrarea unor functionalitati de partajare a rezultatelor pe platforme de socializare
6. **Compatibilitate multiplatform:** extinderea aplicatiei pentru a rula pe mai multe sisteme de operare, precum iOS, utilizand tehnologii de multiplatforming(ex: Kotlin Multiplatform)

Bibliografie

- [1] AnTuTu Benchmark. (2023): <https://www.antutu.com>
- [2] Geekbench.(2023). Evaluarea performantelor CPU si GPU: <https://www.geekbench.com>
- [3] 3DMark. (2023). Benchmark pentru performante grafice <https://www.3dmark.com>
- [4] Android Developers – ghid pentru dezvoltatori despre optimizarea aplicatiilor:
<https://developer.android.com>
- [5] ARM Holdings – documentatie privind arhitectura CPU si GPU pentru dispozitive mobile:
<https://www.arm.com>
- [6] T.H. Cormen. C.E. Leiserson, R.L. Rivest, and C. Stein, "Introduction to Algorithms 3rd ed.", 2009